Produto matriz-vetor distribuído por colunas

Matheus Marinho 29 de setembro de 2026

Conta e divisão

Conta: `y[i] = soma(A[i,j] × x[j])`. Nesta versão, cada processo recebe um grupo de colunas de `A` e a parte correspondente de `x`. Ele calcula uma **contribuição parcial para cada uma das `M` linhas**. `MPI_Reduce` com `MPI_SUM` soma essas contribuições e entrega `y` completo ao processo 0.

Colunas extras: `colunas_locais = ceil(N/P)`. O processo 0 guarda a matriz com `P × colunas_locais` posições por linha; as que passam de `N` valem zero. O mesmo vale para `x`. Assim, cada processo recebe um bloco igual, mesmo quando `N` não é múltiplo de `P`.

chamada	palavra-chave para explicar
`MPI_Type_vector(M, colunas_locais, colunas_distribuidas, ...)`	Seleciona um grupo de colunas em cada uma das `M` linhas. O salto entre linhas é `colunas_distribuidas`.
`MPI_Type_create_resized(..., colunas_locais × sizeof(double), ...)`	Faz o próximo bloco do `Scatter` começar nas colunas seguintes da primeira linha. Não muda os valores selecionados.
1º `MPI_Scatter`	Usa o tipo derivado na origem; `a_local` recebe `M × colunas_locais` valores **compactados**.
2º `MPI_Scatter`	Envia o trecho de `x` que corresponde às colunas recebidas.
`MPI_Reduce` / `MPI_SUM`	Soma, posição a posição, os vetores `y_local` de tamanho `M`.

Correção: para `M=5`, `N=3`, saíram `y[0]=6` e `y[4]=18` com 1, 2 e 8 processos. O teste retangular `M=3`, `N=5` passou com 2 e 8 processos.

Linhas × colunas: tempo e memória

Medição: mesmo job, nó exclusivo `r2n09` do NPAD, GCC 8.5.0, Open MPI 4.1.6 e `-O2`. Foram cinco rodadas alternadas por configuração; a tabela traz a **mediana em segundos**. `MPI_Wtime` mede do `Scatter` até o `Gather` ou `Reduce`. A criação dos dados fica fora do cronômetro.

Matriz	Processos	Linhas (T16)	Colunas (T17)	Colunas / linhas
2048^2	1	0,008515	0,009507	1,12×
2048^2	2	0,014877	0,019313	1,30×
2048^2	4	0,021083	0,027979	1,33×
2048^2	8	0,023278	0,023201	1,00×
4096^2	1	0,033708	0,035740	1,06×
4096^2	2	0,040409	0,051417	1,27×
4096^2	4	0,040321	0,081891	2,03×
4096^2	8	0,050097	0,057542	1,15×
8192^2	1	0,131222	0,139849	1,07×
8192^2	2	0,127577	0,179324	1,41×
8192^2	4	0,131396	0,286481	2,18×
8192^2	8	0,123437	0,228917	1,85×

Leitura: colunas foi mais lenta na maioria dos casos. Em $2048^2/8$, os tempos empataram; em $8192^2/4$, colunas levou **2,18 vezes** o tempo. A diferença não cresce sempre com `P`.

Memória e comunicação: em `A`, linhas são contíguas; colunas exigem saltos entre linhas na origem. O `Scatter` compacta os blocos recebidos: **o cálculo local continua contíguo** e reutiliza `x_local` a cada linha. A T17 ainda reduz `M` valores por processo; a T16 reúne só sua parte de `y`. Esses custos ajudam a explicar a diferença, mas não foram cronometrados separadamente. Em $8192^2/4$, colunas variou de 0,267 a 0,350 s: pequenas diferenças de tempo não sustentam uma conclusão.

Código usado

matriz_vetor_colunas_mpi.c · 99 linhas

```c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);

    int processo, processos;
    MPI_Comm_rank(MPI_COMM_WORLD, &processo);
    MPI_Comm_size(MPI_COMM_WORLD, &processos);

    if (argc != 3 || atoi(argv[1]) <= 0 || atoi(argv[2]) <= 0) {
        if (processo == 0) {
            fprintf(stderr, "Uso: %s M N (ambos positivos)\n", argv[0]);
        }
        MPI_Finalize();
        return 1;
    }

    int m = atoi(argv[1]);
    int n = atoi(argv[2]);
    int colunas_locais = (n + processos - 1) / processos;
    int colunas_distribuidas = colunas_locais * processos;

    double *a = NULL;
    double *x = NULL;
    double *y = NULL;
    double *a_local = malloc((size_t)m * colunas_locais * sizeof(double));
    double *x_local = malloc((size_t)colunas_locais * sizeof(double));
    double *y_local = malloc((size_t)m * sizeof(double));

    if (processo == 0) {
        a = calloc((size_t)m * colunas_distribuidas, sizeof(double));
        x = calloc((size_t)colunas_distribuidas, sizeof(double));
        y = malloc((size_t)m * sizeof(double));
    }
    if (!a_local || !x_local || !y_local ||
        (processo == 0 && (!a || !x || !y))) {
        fprintf(stderr, "Processo %d: memoria insuficiente\n", processo);
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    if (processo == 0) {
        for (int j = 0; j < n; j++) {
            x[j] = 1.0;
        }
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                a[(size_t)i * colunas_distribuidas + j] =
                    (double)((i + j) % 10 + 1);
            }
        }
    }

    MPI_Datatype tipo_colunas, tipo_redimensionado;
    MPI_Type_vector(m, colunas_locais, colunas_distribuidas,
                    MPI_DOUBLE, &tipo_colunas);
    MPI_Type_create_resized(tipo_colunas, 0,
                            (MPI_Aint)colunas_locais * sizeof(double),
                            &tipo_redimensionado);
    MPI_Type_commit(&tipo_redimensionado);
    MPI_Type_free(&tipo_colunas);

    MPI_Barrier(MPI_COMM_WORLD);
    double inicio = MPI_Wtime();

    MPI_Scatter(a, 1, tipo_redimensionado,
                a_local, m * colunas_locais, MPI_DOUBLE,
                0, MPI_COMM_WORLD);
    MPI_Scatter(x, colunas_locais, MPI_DOUBLE,
                x_local, colunas_locais, MPI_DOUBLE,
                0, MPI_COMM_WORLD);
```

```
73
74      for (int i = 0; i < m; i++) {
75          double soma = 0.0;
76          for (int j = 0; j < colunas_locais; j++) {
77              soma += a_local[(size_t)i * colunas_locais + j] * x_local[j];
78          }
79          y_local[i] = soma;
80      }
81
82      MPI_Reduce(y_local, y, m, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
83
84      if (processo == 0) {
85          double tempo = MPI_Wtime() - inicio;
86          printf("M=%d N=%d processos=%d tempo=%.6f s y[0]=%.0f y[%d]=%.0f\n",
87                 m, n, processos, tempo, y[0], m - 1, y[m - 1]);
88      }
89
90      MPI_Type_free(&tipo_redimensionado);
91      free(a);
92      free(x);
93      free(y);
94      free(a_local);
95      free(x_local);
96      free(y_local);
97      MPI_Finalize();
98      return 0;
99  }
```